Techniques algorithmiques de recherche dans les tableaux

Dans le développement de jeux vidéo, nous devons constamment chercher, filtrer et traiter des collections d'objets comme les ennemis, les projectiles ou les objets d'inventaire. Plutôt que d'utiliser les méthodes prêtes à l'emploi comme `filter()` et `find()`, nous allons apprendre à implémenter ces algorithmes nous-mêmes avec des boucles `for`. Cette approche nous permet de mieux comprendre le fonctionnement interne et d'optimiser nos algorithmes selon nos besoins spécifiques.

1. Parcours simple - Parcourir tous les éléments

Le parcours simple consiste à visiter chaque élément du tableau pour effectuer une action. C'est l'algorithme de base pour dessiner tous les objets du jeu. Cela permet de s'assurer que chaque élément est traité, par exemple pour le rendre visible à l'écran. La méthode `foreach()` est une alternative, mais ici nous utilisons `for` pour plus de contrôle.

```
let aEnnemis = [
    {nom: "Gobelin", vie: 30, x: 100, y: 200, actif: true, image: new Image(),
src: "gobelin.png"},
    {nom: "Orc", vie: 50, x: 300, y: 150, actif: true, image:new Image(), src:
"orc.png"},
    {nom: "Dragon", vie: 0, x: 500, y: 100, actif: false, image:new Image(), src:
"dragon.png"}
];

// Algorithme : Dessiner tous les ennemis actifs
function dessinerEnnemis() {
    for (let i = 0; i < aEnnemis.length; i++) {
        // Vérifier si l'ennemi est actif avant de le dessiner
        if (aEnnemis[i].actif === true) {
            let ennemi = aEnnemis[i];
            // Charger l'image si ce n'est pas déjà fait
            if (!ennemi.image.src) {
                ennemi.image.src = ennemi.src;
            }
            // Dessiner l'ennemi sur le canvas
            contexte.drawImage(ennemi.image, ennemi.x, ennemi.y);
        }
    }
}
```

2. Choisir un élément de manière aléatoire

Pour choisir un élément au hasard dans un tableau, on peut générer un index aléatoire compris entre 0 et la longueur du tableau moins un. Cela permet de sélectionner un élément de manière équitable.

```
let indexAleatoire = Math.floor(Math.random() * aEnnemis.length);
let ennemiAleatoire = aEnnemis[indexAleatoire];
```

3. Algorithme de filtrage - Créer un nouveau tableau

Le filtrage consiste à parcourir un tableau et créer un nouveau tableau contenant seulement les éléments qui respectent une condition. On passe en revue chaque élément et on teste la condition. Si la condition est vraie, on ajoute l'élément au nouveau tableau.

Il existe une méthode intégrée `filter()`, mais ici nous allons implémenter notre propre version pour mieux comprendre le processus.

```
const aEnnemis = [
    {nom: "Gobelin", vie: 30, x: 100, y: 200, actif: true},
    {nom: "Orc", vie: 50, x: 300, y: 150, actif: true},
    {nom: "Dragon", vie: 0, x: 500, y: 100, actif: false}
];

// Exemple avec filtrage complexe : ennemis dans une zone
function filtrerEnnemisZone(xMin, yMin, xMax, yMax) {
    let aEnnemisZone = [];

    for (let i = 0; i < aEnnemis.length; i++) {
        let ennemi = aEnnemis[i];
        // Condition complexe : dans la zone ET vivant
        if (ennemi.x >= xMin && ennemi.x <= xMax &&
            ennemi.y >= yMin && ennemi.y <= yMax &&
            ennemi.vie > 0) {
            aEnnemisZone.push(ennemi);
        }
    }

    return aEnnemisZone;
}
```

4. Algorithme de recherche - Trouver UN élément

La recherche consiste à parcourir le tableau jusqu'à trouver le premier élément qui correspond à nos critères, puis arrêter la recherche. On utilisera une boucle `for` avec une instruction `break` pour sortir de la boucle dès qu'on trouve l'élément ou l'instruction `return` pour sortir de la fonction.

```
const aEnnemis = [
    {nom: "Gobelin", vie: 30, x: 100, y: 200, actif: true},
    {nom: "Orc", vie: 50, x: 300, y: 150, actif: true},
    {nom: "Dragon", vie: 0, x: 500, y: 100, actif: false}
];

// Algorithme de recherche avec calcul de distance
```

```
function trouverEnnemiPlusProche(positionJoueur) {
    let ennemiPlusProche = null;
    let distanceMinimale = 3000; // Très grande valeur

    for (let i = 0; i < aEnnemis.length; i++) {
        let ennemi = aEnnemis[i];

        // Ignorer les ennemis morts
        if (ennemi.vie <= 0) {
            continue; // Passer au suivant
        }

        // Calculer la distance
        let distance = Math.sqrt(
            Math.pow(ennemi.x - positionJoueur.x, 2) +
            Math.pow(ennemi.y - positionJoueur.y, 2)
        );

        // Si cette distance est plus petite, c'est notre nouveau candidat
        if (distance < distanceMinimale) {
            distanceMinimale = distance;
            ennemiPlusProche = ennemi;
        }
    }

    return ennemiPlusProche;
}
```

5. Algorithmes de tri avec sort()

La méthode `sort()` de JavaScript permet de trier les éléments d'un tableau. Par défaut, elle trie par ordre alphabétique, mais on peut lui fournir une fonction de comparaison personnalisée pour définir l'ordre de tri.

Syntaxe de base

```
// Tri alphabétique par défaut pour les chaînes de caractères selon l'ordre
Unicode
tableau.sort();
```

Tri numérique simple

Problème courant : Par défaut, JavaScript trie selon l'ordre Unicode (alphabétique), même pour les nombres. Cela cause des résultats inattendus avec les valeurs numériques Car les nombres sont convertis en chaînes de caractères pour la comparaison et non comparés en tant que valeurs numériques. On doit donc fournir une fonction de comparaison qui soustrait les deux nombres. Si la soustraction est négative, `a` vient avant `b`; si elle est positive, `a` vient après `b`.

```
let aScores = [1500, 800, 2300, 950, 1200];

// ❌ Tri alphabétique (incorrect pour les nombres)
aScores.sort();
console.log(aScores); // Résultat: [1200, 1500, 2300, 800, 950]

/*
POURQUOI CE RÉSULTAT BIZARRE ?
JavaScript convertit les nombres en chaînes de caractères :
- "1200" commence par "1"
- "1500" commence par "1"
- "2300" commence par "2"
- "800" commence par "8"
- "950" commence par "9"

En ordre alphabétique : "1" < "2" < "8" < "9"
D'où l'ordre : 1200, 1500, 2300, 800, 950
*/
```

Comment fonctionne la fonction de comparaison

La fonction de comparaison reçoit deux éléments (a et b) et doit retourner :

- **Nombre négatif** (ex: -1) : a doit venir **avant** b
- **Zéro** (0) : a et b sont **égaux** (ordre inchangé)
- **Nombre positif** (ex: 1) : a doit venir **après** b

```
// Tri numérique croissant (du plus petit au plus grand)
aScores.sort(function(a, b) {
    return a - b; // Si a < b, retourne négatif → a avant b
});
console.log(aScores); // [800, 950, 1200, 1500, 2300]

// Tri numérique décroissant (du plus grand au plus petit)
aScores.sort(function(a, b) {
    return b - a; // Si b > a, retourne positif → b avant a (donc a après b)
});
console.log(aScores); // [2300, 1500, 1200, 950, 800]
```

Exemples pratiques pour bien comprendre

```
// Tableau de points de vie d'ennemis
let aVieEnnemis = [25, 100, 5, 80, 15];

// Trier pour cibler les ennemis les plus faibles d'abord
aVieEnnemis.sort(function(a, b) {
    return a - b; // Croissant : faibles → forts
});
```

Tri d'objets par propriété

Concept : Quand on a un tableau d'objets (comme des joueurs, ennemis, objets), on veut souvent les trier selon une de leurs propriétés (score, niveau, nom, etc.). Pour cela, on utilise la **notation par point** pour accéder aux propriétés dans la fonction de comparaison.

```
let aJoueurs = [
    {pseudo: "Player1", score: 1500, niveau: 5, tempsJeu: 1200},
    {pseudo: "GamerPro", score: 2300, niveau: 8, tempsJeu: 1350},
    {pseudo: "NoobSlayer", score: 800, niveau: 3, tempsJeu: 900},
    {pseudo: "ProGamer", score: 1200, niveau: 6, tempsJeu: 1100}
];

// Trier par score (décroissant) pour créer un classement
aJoueurs.sort(function(a, b) {
    return b.score - a.score; // b.score - a.score = décroissant
});
```

Tri alphabétique avec localeCompare()

Pour trier des chaînes de caractères, on utilise la méthode `localeCompare()` qui gère correctement les accents et les règles de la langue.

```
// Pour trier des chaînes de caractères, on utilise localeCompare() car elle gère
correctement les accents et les règles de la langue
aJoueurs.sort(function(a, b) {
    return a.pseudo.localeCompare(b.pseudo); // Tri alphabétique A→Z
});

console.log("=== TRI ALPHABÉTIQUE (A→Z) ===");
for (let i = 0; i < aJoueurs.length; i++) {
    console.log((i + 1) + ". " + aJoueurs[i].pseudo);
}
```

Attention : sort() modifie le tableau original

```
let aScoresOriginaux = [1500, 800, 2300];
let aScoresTries = aScoresOriginaux.sort(function(a, b) { return b - a; });

console.log(aScoresOriginaux); // [2300, 1500, 800] - MODIFIÉ !
console.log(aScoresTries);     // [2300, 1500, 800] - Même référence

// Pour conserver l'original, faire une copie
let aScoresOriginauxCopie = [...aScoresOriginaux]; // Copie avec spread
let aScoresTriesSeparement = aScoresOriginauxCopie.sort(function(a, b) { return b
- a; });
```

6. Algorithmes de vérification

Les algorithmes de vérification permettent de tester des conditions sur les éléments d'un tableau. Au lieu d'utiliser `some()` et `every()`, nous implémentons ces vérifications avec des boucles.

```
// Algorithme "Au moins un" (équivalent some)
function auMoinsUnEnnemiVivant() {
    for (let i = 0; i < aEnnemis.length; i++) {
        if (aEnnemis[i].vie > 0) {
            return true; // Dès qu'on en trouve un, on peut s'arrêter
        }
    }
    return false; // Si on arrive ici, aucun n'était vivant
}

// Algorithme "Tous" (équivalent every)
function tousJoueursNiveauMinimum(niveauMin) {
    for (let i = 0; i < aJoueurs.length; i++) {
        if (aJoueurs[i].niveau < niveauMin) {
            return false; // Dès qu'on en trouve un qui ne respecte pas, c'est
faux
        }
    }
    return true; // Si on arrive ici, tous respectaient la condition
}
```